

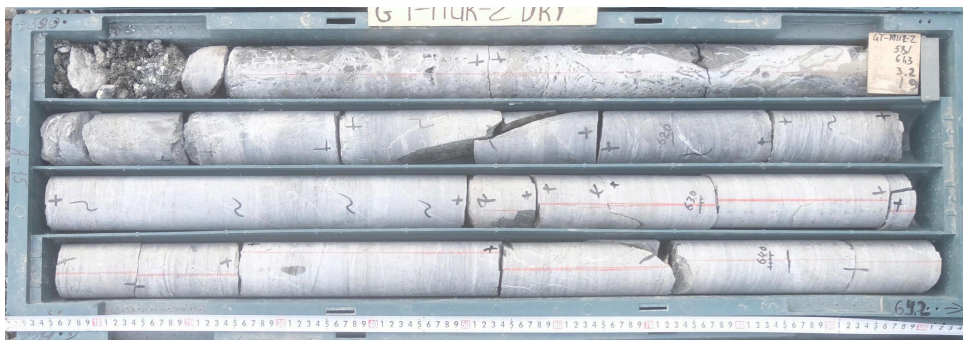
## Что предстоит сделать

Реализовать небольшую веб-страницу, на которой поверх условного изображения керна отображается эллипс — след трещины. Пользователь может мышью уточнять положение центра и правого апекса эллипса.

Задача **не про красивый интерфейс**. Основная цель — корректно реализовать геометрию.

## Предметная область

**Керн** — цилиндрический образец горной породы. На фотографии он расположен горизонтально. След трещины на боковой поверхности керна в ортографической проекции «вид сбоку» выглядит как эллипс.



**Трещина** — плоская поверхность разрыва сплошности, пересекающая керн насквозь. В 3D это **плоскость**, рассекающая цилиндр.

**Апекс** — это *правая крайняя точка эллипса*, то есть точка эллипса с максимальной координатой  $x$ .

Для повернутого эллипса эта точка обычно **не совпадает** с концом большой полуоси.

Пример. Для эллипса с параметрами

```
{ cx: 300, cy: 110, a: 100, b: 50, theta: 45 } // theta в градусах
```

конец большой полуоси находится в точке

```
{ x: cx + a * cos(theta), y: cy + a * sin(theta) } // ≈ (370.7, 180.7)
```

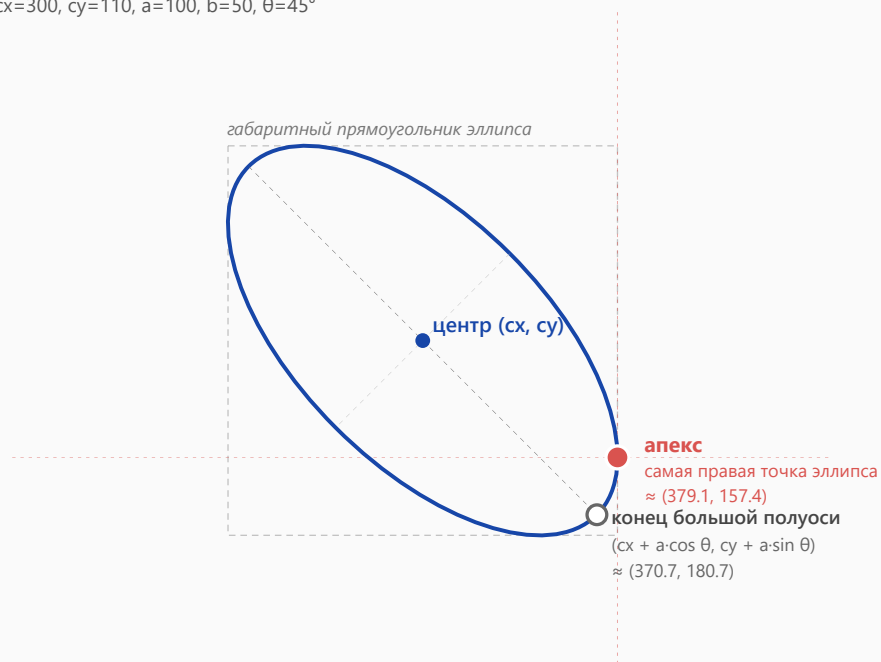
а правый апекс — в точке

```
{ x: ≈ 379.1, y: ≈ 157.4 }
```

Это разные точки.

### Апекс ≠ конец большой полуоси

для эллипса  $sx=300$ ,  $sy=110$ ,  $a=100$ ,  $b=50$ ,  $\theta=45^\circ$



Апекс лежит на правой стороне габаритного прямоугольника. Конец большой полуоси — внутри него.

## Область работы

Достаточно прямоугольника-заглушки:

```
{
  width: 800,
  height: 220,
  coreTop: 30,
  coreBottom: 190,
  coreAxisY: 110
}
```

Керн считается горизонтальным, его ось — линия  $y = 110$ .

## Начальный эллипс

```
{
  cx: 300,
  cy: 110,
  a: 100,
  b: 50,
  theta: 45 // градусы
}
```

Где  $sx$ ,  $sy$  — центр эллипса;  $a$  — большая полуось;  $b$  — малая полуось;  $\theta$  — угол наклона большой полуоси к горизонтали в градусах.

## Что нужно отобразить

На странице должны быть:

1. прямоугольник-керн;
2. эллипс;
3. интерактивная точка центра;

- интерактивная точка правого апекса;
- текущие параметры эллипса:  $s_x$ ,  $s_y$ ,  $a$ ,  $b$ ,  $\theta$ .

## Поведение центра

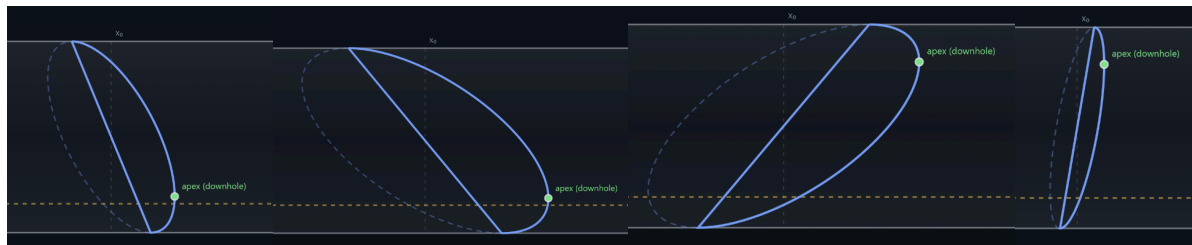
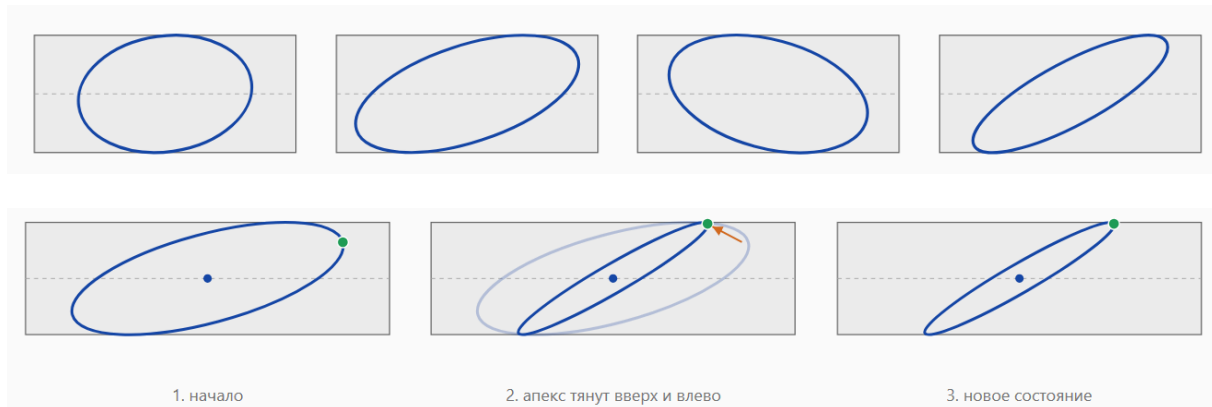
Пользователь перетаскивает центр эллипса. При этом:

- центр движется только вдоль оси керна ( $s_y$  остаётся равным 110);
- форма эллипса не меняется ( $a$ ,  $b$ ,  $\theta$  не меняются);
- апекс пересчитывается автоматически и движется вместе с эллипсом.

## Поведение апекса

Пользователь перетаскивает правый апекс. При этом:

- центр эллипса остаётся неподвижным;
- точка, в которую перетаскили апекс, должна стать новой правой крайней точкой эллипса;
- $a$ ,  $b$ ,  $\theta$  пересчитываются;
- эллипс остаётся геометрически корректным.



Корректно обработайте граничные случаи: попытка увести апекс за границы керна по вертикали, попытка увести центр за пределы видимой части керна и т.п. Что именно делать — на ваше усмотрение, главное, чтобы эллипс никогда не становился геометрически некорректным.

## Технологии

Предпочтительно **Vue 3**. Допустим любой другой стек, если результат убедителен. Внутри — любая отрисовка (Canvas, SVG, WebGL) и любые библиотеки для drag-and-drop или математики.

Использование LLM-ассистентов разрешено.

## README

В README кратко опишите:

- как вы поняли определение апекса;

2. почему апекс не совпадает с концом большой полуоси;
3. какой алгоритм используется для пересчёта эллипса при перетаскивании апекса;
4. какие ограничения и граничные случаи реализованы.

### **Что оценивается**

1. корректность геометрии и различение апекса и конца большой полуоси;
2. устойчивость решения при перетаскивании;
3. ясность объяснения в README;
4. качество кода;
5. способность обосновать решение в разговоре после сдачи.

### **Что не требуется**

- настоящая фотография керна;
- промышленный UI и адаптивная вёрстка;
- ручки сверх центра и апекса (полуоси, угол, масштабирование);
- работа с 3D-геометрией или вычисление углов плоскости трещины;
- продакшн-архитектура.

### **Формат сдачи**

Ссылка на репозиторий или архив с инструкцией по запуску.